

Quorum — developer brief

Everything a software developer needs to build the Round 3 simulation: what the product is, what The Ken asked for, what exists in the repo, what to build tonight, and how the Pine Labs integration works.

Deadline: Sunday 4 October 2026, 11:59 PM IST. No extensions. The submission form (Typeform) has no save button. Write every answer in a separate document first, then paste. Only the team admin (Aabhas) can submit, at the-ken.com/case-competition-2026 → *Submit your solution*.

Repository

`github.com/aabhasamol/nobody-fronts`, branch `claude/vibrant-albattani-sq7g8l`. Python 3.11, FastAPI, pydantic. Run `pip install -r requirements.txt`, then `python -m pytest -q` (57 tests pass) and `./run.sh` (demo UI at `localhost:8000`).

Team

Aabhas, Aditi, Sayan (IIM Calcutta). Opening: *Planning the trip*.

Simulation code

Built on 4 Oct in `sim/`: the Claude agent loop, system prompt, curtain console, Gnani and Pine Labs tools, decision-log export. Run guide: `sim/README.md` and §12.

Contents

1. Quorum in one page
2. The flow and the rules
3. What Round 3 asks for
4. What exists, what's pending
5. Target architecture
6. The agent's tools
7. Pine Labs integration
8. Gnani and Delhivery
9. The simulation script
10. Decision log and submission
11. Build plan for tonight
12. How to run the simulation

Verification status. This brief was written from inside a sandbox that cannot reach the Pine Labs or Delhivery documentation sites (the network proxy blocks them). Every endpoint marked **verify** is copied from our Round 2 research and repository code. Before recording, open the official docs and confirm each path, method and field. The Ken grades "the exact endpoint from their documentation".

1 · Quorum in one page

The problem. When a group of friends plans a trip (a wedding in Goa, a leisure weekend), one person, the organiser, usually ends up booking everything on their own card and chasing everyone for money. People say "yes" in the group chat to be polite, then drop out once money is due.

The product. Quorum is a trip agent that lives in the group's WhatsApp. It separates the *social* decision (a private yes/no vote) from the *money* decision (each payer authorises their own capped amount). It books only once everyone who is in has authorised. **Nobody fronts money for anyone else:** not the organiser, and not Quorum.

The outcome (Round 2 answer, Q1)

Book an approved group trip for every participating payer, including travel from their own city, without requiring any one member to advance another member's share.

Autonomy: L3

Quorum may price, book and rebook a previously approved trip within each payer's own authorised cap and the agreed itinerary. Anything outside that, such as a different hotel, a different date or a cost above someone's cap, goes back to the affected people.

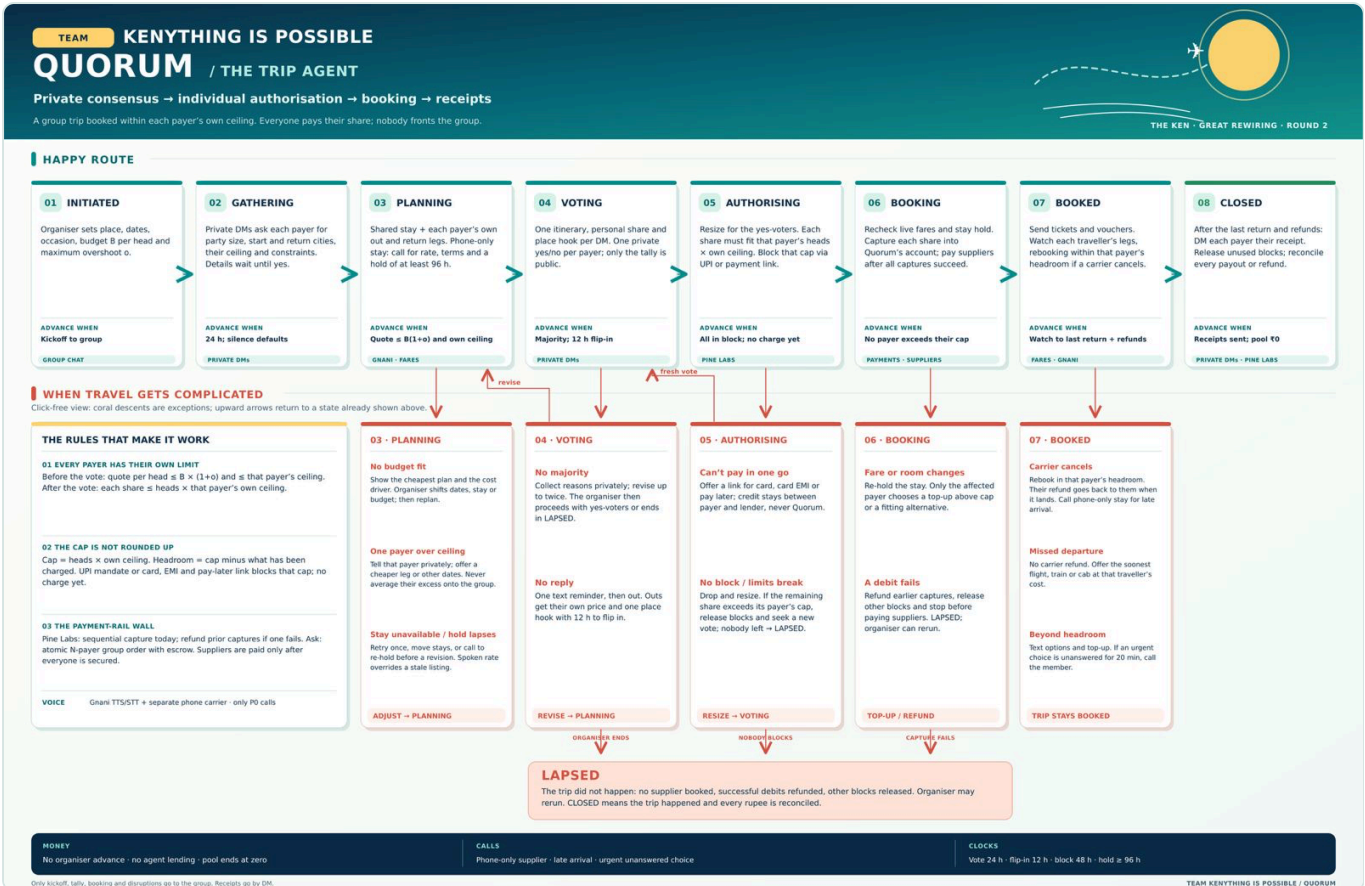
Vocabulary

Term	Meaning
Organiser	Adds Quorum to the group and gives destination, dates, occasion, a rough budget B per head and a maximum overshoot o (e.g. ₹20,000 and 10%). Never pays for anyone.
Payer / member	A person in the group with a phone and a way to pay. A payer may cover several travellers (their <i>party</i> , or <i>heads</i>).
Ceiling	A payer's own per-head maximum, given privately. If they stay silent, it defaults to the organiser's limit $B \times (1+o)$.
Share	What one payer owes: (their own travel legs + their split of the shared stay + pre-booked essentials) $\times$ heads.
Cap	heads $\times$ own ceiling, not rounded up. This is what the payer blocks after the vote. Headroom = cap – amount charged.
Block	A UPI mandate, card pre-authorisation or payment-link payment for the cap. Nothing is charged until booking.
Pool	Quorum's merchant account at Pine Labs. Captures land here; suppliers are paid from here; it must end at ₹0.
Phone-only stay	A homestay or guesthouse with no online inventory. The agent calls it (via Gnani speech) for availability, rate, terms and a hold of at least 96 h.
P0 / P1 / P2	P0 = a phone call (only three reasons are allowed); P1 = a text now; P2 = a text that can wait.

Channels: who hears what

- **Group chat** gets only four kinds of post: the kickoff, the vote tally, "booked", and disruptions.
- **Private DMs** get everything personal: constraints, each payer's own price, the vote, the payment link, receipts. A "no" is never named in the group.
- **Calls (P0 only):** (1) a phone-only stay, for availability and a hold; (2) a late arrival at a phone-only stay; (3) a member who hasn't answered an urgent disruption text after 20 minutes. Members are never called to chase a vote or a payment.

2 · The flow and the rules



The final flow from the Round 2 submission (also in `docs/quorum-flow-final.webp` and the interactive `docs/quorum-trip-flow.html`). The engine in `app/engine.py` implements exactly these eight states plus LAPSED.

The three rules that make it work

1 Every payer has their own limit. Before the vote: each payer's quote per head $\leq B \times (1+o)$ and $\leq$ their own ceiling. After the vote: each share $\leq$ heads $\times$ that payer's own ceiling. One payer's excess is never averaged onto the others, and the group never learns whose ceiling broke.

2 The cap is not rounded up. Cap = heads $\times$ own ceiling. Headroom = cap – charged. A UPI mandate, card hold or pay-later link blocks the cap; nothing is charged yet.

3 The payment-rail wall. Pine Labs has no all-or-nothing capture across N payers, so the agent captures payer by payer and refunds earlier captures if one fails. Suppliers are paid only after every capture succeeds.

Clocks

Window	Length
Gathering (private DMs)	24 h; silence = defaults
Vote	24 h; one text reminder at 12 h
Flip-in (no-voters can join)	12 h
Block (authorise)	48 h default; organiser sets it
Phone-only hold	≥ 96 h (24 + 12 + 48 = 84 h, plus slack)
Disruption escalation	20 min unanswered → call

Money invariants

- The organiser never advances money; Quorum never lends.
- The pool never goes negative and ends at ₹0.
- No payer is charged above their cap without a fresh approval (a top-up).
- Revisions after a failed vote: at most two. Then the organiser decides (book for the yes-voters only, or close).

3 · What Round 3 asks for

Source: Adhavan RK (The Ken), "You're shortlisted to Round 3" (30 Sep 2026) and "Round 3 submission is open" (1 Oct 2026). Paraphrased closely; quotes are verbatim.

Part 1: run a simulation of the agent ("Wizard of Oz")

The agent is an AI model (Claude or Gemini) "following a system prompt you write, and it makes every decision on its own". Wherever the model can connect to something directly (Gmail, WhatsApp, Google Sheets), connect it. Everything else is played by a team member "behind the curtain":

- **Gnani:** "run the audio or text through Gnani's API yourself, and paste back exactly what Gnani returns."
- **Pine Labs and Delhivery:** "reply with the response their documentation says they'd send."
- **Anything else from the outside world** (a voice note arriving, the clock hitting 5 PM): a team member feeds it in. "It must come from a real source. If no real source produces it today, your agent can't have it."

The curtain never helps the agent think. "If your agent needs someone to tell it what to do next, it isn't an agent yet."

The recording

- Screen-record from the first thing that happens to the moment the outcome is achieved. **5 minutes at most**; fast-forward model thinking.
- It must show **every input arriving, every call to Gnani, Pine Labs or Delhivery with the response, and every message the agent sends to a person.**
- Share the link, the **system prompt** and the **name of the model.**

Questions to answer (Part 1)

1. The story of one person using the agent: named, dated, in order. **≤ 100 words.**
2. **Every decision in the recording, in order.** For each: when (date and time), what the agent received, where it came from (connector and real source), what it decided, why (the system-prompt rule), what it did or said and to whom (word for word), and through what connector.
3. What the agent knows on day one, who gives it that information, and how.
4. For every Pine Labs, Delhivery or Gnani call: **the exact endpoint** from their docs, plus request and response. Up to **3 imagined capabilities**, listed separately: partner, endpoint, and the data that partner already holds which makes it possible.
5. A mockup of every screen or conversation where the agent meets a person (WhatsApp chat, call script, app screen, notification).
6. Score each rail out of 10 on how agent-ready it is, with one reason each.

Part 2: build the business around it

1. Where the agent lives and how a customer starts using it (own app, a feature in a named app, a WhatsApp number...).
2. Which company should build it, why them, what they have that nobody else does, and why they haven't built it.
3. Who they sell to, how they make money, what customers do today and what that costs.
4. **How it gets its first 1,000 users** ("the question we care about most"), and what keeps them after three weeks.
5. **A landing page** for the agent, made the way the named company would, with a sign-up CTA. Share the link.

developer task
6. What could kill this agent in its first year.

4 · What exists, what's pending

Repository map

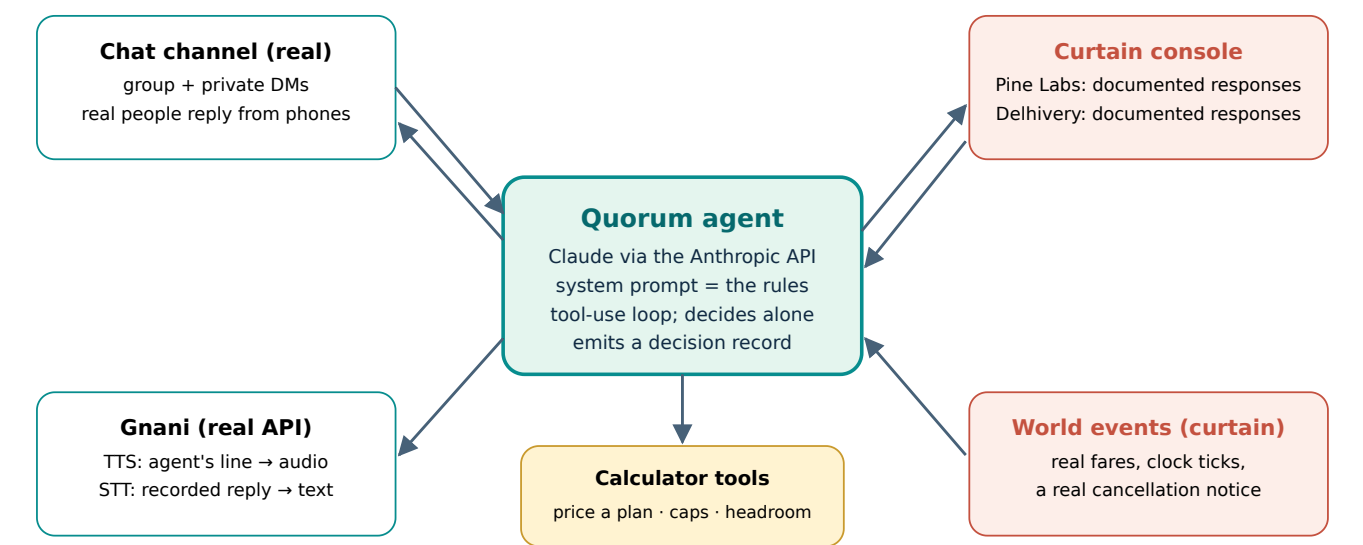
Path	What it is	Status
app/engine.py	The state machine: INITIATED → ... → CLOSED, every unhappy path, the P0 call rule, clocks. Deterministic Python.	done
app/models.py	Trip, Member, Plan, Leg, Stay, Authorisation, instruments (UPI Reserve, UPI OTM, card hold, prepaid link, BNPL).	done

app/messages.py	Every sentence the agent sends. Useful as tone reference for the system prompt.	done
app/rails/mock.py	Offline mocks for voice, payments and logistics (fares, stays, activities).	done
app/rails/gnani_speech.py , gnani.py	Real Gnani TTS + STT via <code>api.vachana.ai</code> ; a supplier call run on recorded replies; budget cap on billable calls; key in <code>app/keys.py</code> .	works
app/rails/pinelabs.py	Pine Labs adapter: token, payment link with pre-auth, status, capture, cancel, refund, offer discovery; payouts recorded as instructions.	unverified
app/main.py , static/index.html	FastAPI + demo UI with group chat, DMs and rail log; buttons drive each member.	done
demo.py , docs/demo_transcript.txt	Three terminal runs (wedding, leisure, payment wall).	done
docs/round2-final.md	The submitted Round 2 answers. Read this first.	reference
sim/agent.py , sim/system_prompt.md	Round 3: Claude tool-use loop (one event in, the model decides until done) and the rules R1-R26 it follows; operator console.	built
sim/tools.py , sim/world.py , sim/curtain/templates.json	Round 3: tools (chat, Gnani, phone line, Pine Labs, Delhivery, fares, stays, calculators), curtain console with documented-response templates, sim clock, run log.	templates to verify
sim/channels.py , sim/export.py	Round 3: console or Telegram channel; exports decisions.md, rail_calls.md, transcript.md.	built

Gap against Round 3

#	Pending	Why it matters	Priority
1	LLM agent + system prompt + tool loop	The Ken requires the <i>model</i> to make every decision. Built in <code>sim/</code> ; needs a rehearsal with a real API key.	built
2	Curtain console for Pine Labs, Delhivery and world events	A teammate answers each Pine Labs/Delhivery call with the documented response, on screen, and injects real-world inputs. Built ; templates must be replaced with the docs' samples.	built
3	Endpoint verification	All Pine Labs paths and both Delhivery paths in <code>sim/curtain/templates.json</code> are unverified. A person with docs access must check them.	blocking
4	Real chat channel	Console works now; Telegram channel is built but needs a bot token and chat ids in <code>sim/config.json</code> .	config
5	Real sources for inputs	Fares, stay listings, flight status and the cancellation event are mock. Each must come from a real source (e.g. a teammate pastes a real fare from an airline/OTA site with a screenshot).	blocking
6	Move the trip dates	The simulation takes dates from the organiser's message (rehearsal uses 20-24 Nov 2026). The old engine demo still says 2-6 Oct; it isn't used in the recording.	done
7	Decision-log export	Every action tool carries a decision (rule id, trigger, state); <code>sim/export.py</code> writes the table. Built .	built
8	Landing page (Part 2 Q5)	Static page with a sign-up CTA, deployed with a public link.	high
9	Mockups of every touchpoint	WhatsApp group, each DM type, supplier call script, escalation call, Pine Labs hosted page, receipt.	high
10	Engine leftovers	Per-payer receipt at CLOSED (blocked · charged · refunded · released) not built; group post after a missed flight says "cancelled by the carrier" (wrong); UPI Reserve Pay is modelled up to ₹1 lakh but our own Round 2 text says its cap is ₹10,000.	later

5 · Target architecture for the simulation



The design principle: tools return facts, the model decides

The Ken's test is whether the model makes every decision. So:

- **The rules live in the system prompt**, each with an ID (R1, R2, ...), so each decision can cite the rule it followed. Port them from `app/engine.py` and §2.
- **Tools never decide.** A tool sends a message, calls Gnani, calls (curtain-played) Pine Labs or Delhivery, or computes arithmetic such as a share, a cap or headroom. Computing numbers is fine; choosing what to do with them is the model's job.
- **The Python engine becomes the referee, not the brain.** After a rehearsal run, replay the same inputs through `app/engine.py` and compare. Where the model's decision differs from the engine's, fix the system prompt before recording.
- **Event-driven loop.** Each inbound event (a chat message, a curtain response, a clock tick) is appended to the conversation as a user turn. The model replies with tool calls until it has nothing more to do, then waits for the next event.

Channel options (decide in the first 15 minutes)

Option	Setup	Fit
A. WhatsApp Cloud API test number	Meta developer app; test number messages up to 5 verified recipients; business-initiated messages need a template until each member messages first (24 h window). Bots cannot join existing WhatsApp groups.	Closest to the product, but the riskiest setup for tonight. Group posts would need a workaround.
B. Telegram bot	BotFather token in minutes; the bot can sit in a real group and DM each member.	Fastest real channel with a true group + DMs. Say in the submission that production is WhatsApp; mockups show WhatsApp.
C. Our own web chat (<code>static/index.html</code>)	Already built.	Model connects directly to our app, but judges may see it as less real. Use only as fallback.

Recommendation: B for the recording, with WhatsApp mockups for Q5. Optionally mirror the ledger to a Google Sheet (a direct connector The Ken names).

6 · The agent's tools

Defined in `sim/tools.py` as Anthropic Messages API tools; the loop is in `sim/agent.py` (model `claude-opus-5-5`, adaptive thinking with summaries shown, effort `high`, prompt caching, server-side refusal fallback). Every

action tool takes a required `decision` object (`decided` , `rule_id` , `trigger` , `state`) that becomes one row of the decision log. Tools act and report facts; they never decide.

Tool	What happens	Played by
<code>send_group</code> , <code>send_dm</code>	Post to the group (kickoff, tally, booked, disruptions) or DM one member.	real channel (console or Telegram)
<code>log_decision</code>	A decision with no other action (e.g. "wait for the deadline").	log
<code>search_fares</code> , <code>search_stays</code>	Curtain pastes what a real airline/OTA site or Google Maps shows now; keep screenshots.	curtain (real source)
<code>delhivery_geocode</code> , <code>delhivery_distance</code>	Venue and stay coordinates; road distance and drive time.	curtain (documented response)
<code>gnani_tts</code>	POST <code>https://api.vachana.ai/api/v1/tts/inference</code> ; saves the WAV in the run folder.	Gnani (real)
<code>place_call</code>	Plays the TTS audio; the operator gives the path of the callee's recorded reply, or "none" for no answer. Only P0 reasons accepted by schema.	curtain (phone line)
<code>gnani_stt</code>	POST <code>https://api.vachana.ai/stt/v3</code> ; Gnani's transcript returned verbatim.	Gnani (real)
<code>pinelabs_create_payment_link</code>	Builds the documented body (cap in paisa, <code>pre_auth</code> , card/UPI/BNPL, <code>expire_by</code>). Token call happens automatically first.	curtain, or UAT with <code>SIM_PINELABS=uat</code>
<code>pinelabs_get_payment_link</code>	Status, order id, method. (Or the operator plays the webhook: <code>/webhook Name AUTHORIZED CARD .</code>)	curtain / UAT
<code>pinelabs_capture</code> , <code>_cancel</code> , <code>_refund</code> , <code>_payout</code>	Capture the share; release a block; refund on a failed group capture; pay a supplier from the pool. The operator console warns if a capture exceeds the authorised amount or a payout exceeds the pool.	curtain / UAT
<code>quote_plan</code> , <code>caps</code>	Arithmetic only (same formulas as <code>app/models.py</code> , tested against the engine's numbers).	code

The system prompt

`sim/system_prompt.md` holds the outcome, how the agent works, and rules **R1-R26**: channels and tone (R1-R2), gathering (R3-R5), planning (R6-R9), voting (R10-R13), authorising (R14-R16), booking (R17-R20), after booking (R21-R24), calls (R25) and facts (R26). `build_system()` appends day-one knowledge (group, roster, organiser, place hooks from `app/lore.py`). Each run saves the exact prompt as `system_prompt.md` in its run folder; submit that file.

7 · Pine Labs integration

Pine Labs Online (the "Plural" v2 APIs). UAT base <https://pluraluat.v2.pinepg.in>; developer keys from dashboard.v2.pluralonline.com. In the simulation a teammate plays Pine Labs by pasting the documented response. Still build each real request body in code: it proves the integration and makes the switch to live UAT a config change (`QUORUM_PAYMENTS=pinelabs`).

Every path below is `verify` . Open developer.pinelabsonline.com and confirm method, path, field names and the exact response shape. Copy the documented sample response into the curtain console's template library. Note any difference here and in `app/rails/pinelabs.py` .

How Quorum uses Pine Labs, step by step

#	When	Call	Purpose
1	Start of AUTHORISING	POST <code>/api/auth/v1/token</code>	OAuth client-credentials token (Bearer for every call below).
2	Per payer, after the vote	POST <code>/api/pay/v1/paymentlink</code>	Hosted link for the payer's cap (paisa), <code>pre_auth</code> on, card + UPI (+ BNPL if enabled), <code>expire_by</code> = block deadline. Link URL goes in the payer's DM.
3	Payer pays on the hosted page	Webhook to <code>callback_url</code> , or GET <code>/api/pay/v1/paymentlink/{payment_link_id}</code>	Status (e.g. authorised/blocked), the <code>order_id</code> behind it, and the method used (card, UPI...).
4	(Optional) EMI quote	POST <code>/api/pay/v1/offers/discovery</code>	EMI tenures for the cap, shown in the DM.
5	BOOKING, per payer	<code>.../api/pay/v1/orders/{order_id}/capture</code> (method: verify; repo uses POST)	Capture exactly the share ($\leq$ cap). One capture per card hold or UPI OTM.
6	A capture fails	POST <code>/api/pay/v1/refunds/{order_id}</code>	Refund each earlier capture; then LAPSED.
7	Release unused blocks / dropouts	<code>.../api/pay/v1/orders/{order_id}/cancel</code>	Releases a hold or the uncaptured balance.
8	All captures succeeded	Payouts API (path: verify)	Pool → homestay and other suppliers (IMPS/NEFT/UPI to registered beneficiaries).

Request: create the block (payment link)

```
POST {base}/api/pay/v1/paymentlink
Authorization: Bearer <access_token>   Request-ID: <uuid>   Request-Timestamp: <ISO-8601>
{
  "amount": { "value": 4400000, "currency": "INR" },           // Aabhas: 2 heads x Rs 22,000 = Rs 44,000
  "description": "Quorum: your share of the trip, blocked now, debited only when everyone is in",
  "expire_by": "2026-10-07T20:00:00Z",
  "allowed_payment_methods": ["CARD", "UPI", "BNPL"],
  "pre_auth": "true",
  "merchant_payment_link_reference": "quorum-plan_8c28-m_aabhas-3f9alc",
  "customer": { "first_name": "Aabhas", "last_name": "Amol", "mobile_number": "98300xxxxx",
    "country_code": "91", "email_id": "...", "merchant_customer_reference": "quorum-m_aabhas" },
  "callback_url": "https://<tunnel>/webhooks/pinelabs",
  "merchant_metadata": { "source": "quorum", "plan": "plan_8c28", "member": "m_aabhas" }
}
```


Request: capture the share

```
{base}/api/pay/v1/orders/{order_id}/capture
{ "merchant_capture_reference": "cap-7c1e9a20b4",
  "capture_amount": { "value": 3996000, "currency": "INR" } } // share Rs 39,960 <= cap Rs 44,000
```

Known walls (put these in the rail score and imagined capabilities)

- No atomic capture across N payers and no group escrow: capture is sequential with compensating refunds, and refunds take days to land.
- A card hold lives 5-7 days and takes one capture; a UPI one-time mandate takes one debit. Only UPI Reserve Pay keeps headroom after the first debit, and its published cap is ₹10,000, too low for a ₹20,000+ share.
- Captured money must settle before it can pay suppliers; supplier fulfilment needs a settlement arrangement or must wait.

8 · Gnani and Delhivery

Gnani (real API calls on screen)

Our key is a Vachana (Gnani's speech platform) key in `app/keys.py`, scoped to TTS and STT and capped by `CallBudget` (40 billable calls; override with `GNANI_CALL_BUDGET`). Free credits: ₹1,000 (Gnani email, 15 Sep).

Verb	Endpoint (from <code>app/rails/gnani_speech.py</code>)	Body	Returns
Text to speech	POST <code>https://api.vachana.ai/api/v1/tts/inference</code>	JSON {text, voice, model, audio_config}	audio bytes
Speech to text	POST <code>https://api.vachana.ai/stt/v3</code>	multipart {audio_file, language_code}	{success, transcript}

The supplier call in the simulation. The agent decides to call the phone-only homestay (R4) and writes its question. TTS turns it into Hindi audio, which is played. A teammate playing the homestay owner records a real spoken reply (rooms, rate, refund terms, hold). STT transcribes it. The agent reads the fields from Gnani's exact transcript. Run `python scripts/speech_demo.py` to see the existing loop (≈ ₹2 per run). There is no outbound telephony; the phone line is a separate seam (Exotel/Twilio) and is not needed for the simulation.

Delhivery (curtain-played)

Delhivery moves parcels; nothing in a group trip is a parcel. What Quorum uses is **Delhivery Maps**: geocoding the venue and the stays, and road distance / travel time (venue ↔ stay, airport ↔ stay). For a wedding, stays are ranked by distance to the venue and the arrival buffer must be real. Today `Stay.km_to_venue` is hand-written; there is no Delhivery adapter.

- **To build:** a `delhivery_distance` tool. The harness shows the request built per Delhivery's Maps docs (geocode, then distance matrix / routing); the curtain pastes the documented response with real coordinates (e.g. from a maps lookup of Assagao and the homestay).
- **Verify on Delhivery's developer portal:** exact Maps endpoint paths, auth header, and response fields. verify
- Delhivery has no flight fares, rooms or travel booking inventory. Don't claim them.

Imagined capabilities (≤ 3): suggestions for the team

Partner	Endpoint (imagined)	What it does	Data the partner already holds
---------	---------------------	--------------	--------------------------------

Pine Labs	POST /api/pay/v1/group-orders	One group order holding N independently approved payer authorisations to one deadline; captures all or none; releases all on failure; escrow until fulfilment.	Each payer's pre-auth / mandate status, the merchant's orders, settlement accounts.
Pine Labs	POST /api/settlements/v1/orders/{order_id}/disburse	Pay a supplier directly from a captured, not-yet-settled order, so booking needn't wait for settlement.	Captured order funds and the merchant's settlement ledger.
Delhivery	GET /maps/v1/access-check?lat=..&lng=..&vehicle=tempo_traveller	Can this vehicle reach this address's entrance? (Lane width, failed-access history.)	Years of delivery attempts per address and pincode with vehicle type and failure reasons.

9 · The simulation script

Scenario. Priya's wedding in Assagao, Goa, moved to a future date (suggested 20–24 Nov 2026). The cast from `app/scenario.py`: Sayan (organiser, Kolkata, ceiling ₹24k), Aabhas (Kolkata, pays for 2, ₹22k), Aditi (Bengaluru out, Mumbai back, ₹20k), Riya (Delhi, ₹23k), Karan (Mumbai, silent → ₹22k default). Budget ₹20,000/head, overshoot 10%. Real people play the members from their own phones. Real fares are pasted by the curtain from a real site at recording time; numbers will differ from the mock, which is fine.

Outcome to reach on camera: BOOKED: every payer's share captured, suppliers paid, tickets in DMs, "Booked" in the group. A disruption beat is optional if time remains.

Run of show (target ≤ 5:00 after fast-forwarding)

Time	Input arrives (source)	Agent does (connector)
0:00	Sayan's kickoff in the group (real chat)	Posts the kickoff; DMs each payer six questions with a default and deadline.
0:30	Four replies; Karan silent (real chat). Clock tick 24 h (curtain).	Acknowledges privately; applies Karan's defaults (R2).
1:00	Real fares and stays (curtain, screenshots); venue distance (curtain-played Delhivery)	Ranks stays by distance to venue; picks the phone-only homestay.
1:20	—	Gnani TTS of the supplier question → plays. Teammate's recorded reply → Gnani STT → agent reads rate ₹3,200, refund terms, 96 h hold (R4).
1:50	—	Prices the plan (calculator); checks each payer against B(1+o) and own ceiling (R3); DMs each their own all-in and a place hook.
2:20	Votes by DM: 4 yes, Karan no (real chat)	Posts only the tally; re-sizes for 5 heads; checks shares against caps (R9); one private flip-in offer to Karan.
2:50	—	Pine Labs token + 4 payment links (curtain responses on screen); DMs each payer their link, cap and headroom.
3:30	Payment status per payer (curtain: documented webhook / status response)	Marks each blocked; when all four are in, starts BOOKING (R10).
3:50	Live fares re-check (curtain)	Captures each share (4 × curtain responses); pays the homestay from the pool (payout response).
4:30	—	Tickets and vouchers by DM; "Booked" in the group. Outcome reached.

Curtain operator rules

- Answer only what the called system would answer. Never hint at what the agent should do next.
- Pine Labs/Delhivery answers come from the documented sample response, with values edited to match the request (amounts, ids, timestamps). Keep the templates in `sim/curtain/`.
- World inputs need a real source: keep a screenshot of every fare, listing, and the cancellation notice if you run a disruption.
- Gnani answers are never typed by hand: the harness pastes Gnani's exact output.

What the agent knows on day one (Q3 draft)

- **The system prompt** (rules, tone, P0 call list): written by the team.
- **The group roster and phone numbers**: from the chat when the organiser adds Quorum.
- **The trip brief** (place, dates, occasion, B, o): from the organiser's kickoff message.
- **Place hooks** (film locations, food stops): a curated table from real sources (`app/lore.py`), loaded by the team.
- **Rail credentials** (Pine Labs merchant, Gnani key): configured by the team.
- It does **not** know members' ceilings, cities or dates until each tells it privately.

10 · Decision log and submission artefacts

Decision log format (Q2)

The harness writes one row per `log_decision` call and exports Markdown/CSV for pasting into the form. Example row:

When	Received	From	Decided	Why	Did / said, to whom	Through
Sun 4 Oct, 20:41 IST	Karan's vote: "No, can't get leave that week"	Chat connector · Karan's phone	Count as no; offer one private flip-in with his own price, 12 h	R5, R7	DM to Karan: "The others are going, so one fact in case it changes anything... ₹12,200 a head... Say yes by Mon 08:41"	Chat (DM)

Submission artefacts and owner

Artefact	Source	Owner
Recording link (≤ 5 min)	Screen capture of chat + harness panels; fast-forward thinking	dev + team
System prompt + model name	sim/system_prompt.md ; model id from the harness log	dev
Story (≤ 100 words)	From the recording	team
Decision table	Harness export	dev
Day-one knowledge	\$9 draft	team
Endpoints + request/response	Harness log (every rail call is recorded with both sides)	dev
≤ 3 imagined capabilities	\$8 draft	team
Mockups of every touchpoint	Screenshots from the run + WhatsApp-style mocks; supplier and escalation call scripts	dev + team
Rail scores	\$8 draft	team
Part 2 answers	Round 2 picked MakeMyTrip (Q8); revisit	team
Landing page link	Static page with sign-up CTA, deployed publicly	dev

Rail scores (draft, for the team to confirm)

Rail	Score	One reason
Gnani	7/10	TTS and STT are callable today with a key; the call-to-booking handoff (telephony, interruptions, confirmation) is ours to build.
Pine Labs	6/10	Links, pre-auth, capture, refunds and payouts exist; no atomic N-payer capture and settlement timing blocks instant supplier pay.
Delhivery	4/10	Maps give distances; nothing about rooms, fares or whether a vehicle can reach a homestay.

Touchpoints to mock (Q5)

1. Group: kickoff, vote tally, booked, disruption.
2. DM: gathering questions; vote with own price; "over your ceiling" heads-up; flip-in offer; payment link with cap and headroom; blocked acknowledgement; tickets; top-up request; receipt.
3. Supplier call script (Hindi), late-arrival call, member escalation call.
4. Pine Labs hosted payment page (card / UPI / EMI / pay later).

11 · Build plan for tonight

In order. Anything not done by 22:30 IST gets cut; the form needs about an hour of pasting.

#	Task	Done when	Status
1	Agent loop, tools, curtain, decision log, export (sim/)	64 tests pass offline	done
2	System prompt with rule ids	sim/system_prompt.md	done
3	Set ANTHROPIC_API_KEY; rehearse with --script sim/scenario/rehearsal.txt	Run reaches BOOKED without operator hints; fix the prompt where it doesn't	30-40 min
4	Verify Pine Labs + Delhivery endpoints; paste the docs' sample responses into sim/curtain/templates.json	No verified: false on endpoints used in the run	30 min, parallel
5	Record the homestay owner's reply (WAV ≤ 60 s, Hindi): rooms, ₹/night, refund terms, 96 h hold	File ready to type at the phone line: prompt	10 min
6	Look up real fares (Kolkata, Bengaluru→Mumbai, Delhi ↔ Goa for the trip dates) and real stays; screenshot	Ready to paste at the curtain prompt	20 min, parallel
7	(Optional) Telegram: bot, group, members press Start, ids in sim/config.json	Bot DMs each member	15 min
8	Record ≤ 5 min; /export	Video link + decisions.md + rail_calls.md	20 min
9	Landing page with sign-up CTA (another teammate)	Public link	45 min, parallel

Acceptance checks before you record

- Every decision row cites a rule id from the system prompt; no row says "operator told it".
- Every Pine Labs/Delhivery/Gnani call shows both request and response on screen.
- No personal number (ceiling, share, a "no") appears in the group chat.
- No payer is ever asked to block more than heads × their own ceiling; the pool ends at ₹0.
- The recording starts at the first input and ends at the outcome.

Honesty notes for the submission. Gnani calls are real; Pine Labs and Delhivery responses are curtain-played from their documentation; fares and listings were looked up live on real sites (keep screenshots); the chat channel used for recording is named. Our Round 2 text says "Our recorded-reply demo is not evidence of a live supplier booking". Keep that tone.

12 · How to run the simulation

```
pip install -r requirements.txt anthropic
export ANTHROPIC_API_KEY=...           # or: ant auth login
python -m pytest -q                     # 64 tests, offline
python -m sim.agent                     # type the world in at world>
python -m sim.agent --script sim/scenario/rehearsal.txt # chat lines from a file
```

The operator's console

Type at world>	The agent receives
@Sayan group: Priya's wedding in Assagao...	Sayan's post in the group
@Riya: Yes	Riya's DM
/advance 24h	a clock tick (30m, 6h, 1d)
/webhook Aabhas AUTHORIZED CARD	Pine Labs' webhook for Aabhas's latest payment link (also FAILED, EXPIRED)
/event IndiGo SMS 6E 523 on 20 Nov is cancelled	a real-world notice with its source

<code>/status</code> , <code>/export</code> , <code>/quit</code>	(operator only) ledger; write the Markdown tables; stop
--	---

On every Pine Labs, Delhivery, fare or stay call the curtain prompt appears: `[Enter]` `send` · `e` `edit` · `p` `paste` `JSON` · `x` `documented error` · `x` on a capture plays the documented failure and exercises R18 (refund earlier captures, LAPSED).

Environment switches

Variable	Effect
<code>SIM_CHANNEL=telegram</code> + <code>TELEGRAM_BOT_TOKEN</code>	Real group and DMs through a Telegram bot (ids in <code>sim/config.json</code>)
<code>SIM_PINELABS=uat</code>	Send the same requests to Pine Labs UAT (<code>PINELABS_CLIENT_ID</code> , <code>_SECRET</code> , <code>_MERCHANT_ID</code> , <code>PINELABS_BASE</code>)
<code>PINELABS_CAPTURE_METHOD</code> , <code>PINELABS_CANCEL_METHOD</code>	Default <code>PUT</code> ; set to what the docs say
<code>QUORUM_MODEL</code> , <code>QUORUM_EFFORT</code>	Default <code>claude-opus-5-5</code> , <code>high</code>
<code>SIM_CURTAIN=auto</code>	Accept every template unedited (rehearsal only, never for the recording)

What a run produces (`sim/runs/<stamp>/`)

- `decisions.md` : Part 1 Q2 table, one row per decision.
- `rail_calls.md` : Part 1 Q4, every call with endpoint, request and response, and who played it.
- `transcript.md` : every message the agent sent (story, mockups).
- `system_prompt.md` : the exact prompt (submit it, with the model name).
- `*.wav` : Gnani's audio and the recorded replies.

Recording layout: terminal on one side, chat on the other. Cyan = input arriving, magenta = call to a rail, green = response, yellow ♦ = decision, bold = message to a person, dim = the model's summarised reasoning.